

## Corso di Programmazione Web – Esercitazione – 22 Maggio 2026

Tutto il lavoro deve essere svolto individualmente.

Rispondere a ogni domanda in modo chiaro, utilizzando schizzi, tabelle o descrizioni dove necessario. Non scrivere codice, ma fornire dettagliate progettazioni API. Scrivere nome, cognome, numero di matricola nel foglio dove si svolge l'esame. Per risparmiare spazio e tempo, si può usare il seguente formato per descrivere le API:

```
{ idEsempio : int : 1           // assegnato dal server
  campoEsempio : str : "esempio" // obbligatorio
  altroCampo : int : 1           // opzionale, minimo = 0, massimo = 10 }
```

### Sistema di Gestione Turni di Volontariato

Progettare un'applicazione web che consenta agli utenti di organizzare e partecipare ad attività di volontariato nella propria città (es. pulizia parchi, raccolta fondi, supporto anziani, ecc.).

#### API richieste

- API 1: Inserire una nuova attività di volontariato
- API 2: Consultare la lista di attività disponibili
- API 3: Consultare il dettaglio di una delle attività disponibili

#### Requisiti dell'applicazione

Ogni attività ha: idAttività, titolo, descrizione, luogo, data, orainizio, oraFine, categoria, idOrganizzatore. Gli utenti sono identificati da un idUtente già esistente nel server (non serve API di registrazione).

#### Specifiche tecniche

- idAttività viene assegnato dal server.
- Si assuma che l'utente organizzatore sia assegnato dal server (l'utente ha effettuato il login).
- orainizio e oraFine sono nel formato HH:MM; oraFine deve essere successiva a orainizio.
- Data è nel formato DD-MM-YYYY.
- Le categorie sono limitate ai valori: "ambiente", "sociale", "educazione", "salute", "altro".

### Domanda 1 – Progettazione delle API (15 punti)

Progettare le tre API descritte sopra. Per ciascuna API:

- 1 punto: endpoint e metodo HTTP
- 2 punti: modello di richiesta/risposta completo con tipi di dato e vincoli (inclusi quali campi sono obbligatori/opzionali, quali sono assegnati dal server, ed eventuali massimi e minimi)
- 1 punto: esempio concreto (richiesta e risposta)

Infine, 3 punti in totale sono assegnati in base alla chiarezza e completezza delle informazioni che complementano il servizio (per esempio, menzione specifica di controlli logici/validazioni server-side).

### Domanda 2 – Validazione e gestione errori (5 punti)

- 1 punto: descrizione dei seguenti codici di errore (404, 422, 500). Elaborare la risposta – non è sufficiente scrivere il "titolo" del codice di errore.
- 2 punti per ogni esempio: fornire 2 esempi concreti (uno per ogni errore) con messaggio JSON e spiegazione, di richieste che generano i codici di errore 404 e 422. Gli errori devono riferirsi chiaramente alle API appena progettate.

### Domanda 3 – Estensione (3 punti)

Attualmente, il server permette a chiunque di inserire una nuova attività. Descrivere quali modifiche sono da applicare per fare in modo che solo gli utenti *amministratori* possano farlo.

## Traccia di soluzione

### Domanda 1 – Progettazione delle API (15 punti)

#### API 1: Inserimento attività

POST /attivit 

idAttivit , titolo, descrizione, luogo, data, oraInizio, oraFine, categoria, idOrganizzatore.

```
{
  "idAttivit ": str : "9adckx92JJs" // assegnato dal server
  "titolo": str : "Pulizia del parco" // obbligatorio, max 100 caratteri
  "descrizione": str : "Raccolta rifiuti" // obbligatorio, max 500 caratteri
  "luogo": str : "Parco della Musica, Cagliari" // obbligatorio
  "data": str : "2026-05-15" // obbligatorio, formato YYYY-MM-DD
  "oraInizio": str : "09:00" // obbligatorio, formato HH:MM
  "oraFine": str : "12:00" // obbligatorio, formato HH:MM
  "categoria": str : "ambiente" // obbligatorio, valori inclusi nella lista permessa
  "idOrganizzatore": int : "1" // assegnato dal server perch  l'utente   loggato
}
```

**Modello di risposta (201 Created/200 OK):**

```
{
  "idAttivit ": str : "9adckx92JJs" // assegnato dal server
}
```

#### API 2: Consultazione attivit 

**Endpoint e metodo HTTP:**

GET /attivit 

**Modello di richiesta:**

Nessun body richiesto.

Esempio di richiesta: GET /attivit 

**Modello di risposta (200 OK):**

```
[
  {
    "idAttivit "      : int : 1
    "titolo"          : str : "Pulizia del parco"
    "luogo"           : str : "Parco della Musica, Cagliari"
    "data"            : str : "2026-05-15"
    "oraInizio"       : str : "09:00"
    "oraFine"         : str : "12:00"
    "categoria"       : str : "ambiente"
    "idOrganizzatore" : int : 7
  },
  { ... altre attivit  ... }
]
```

### API 3: Consultazione singola attività

#### Endpoint e metodo HTTP:

GET /attivita/{idAttivita}

#### Modello di richiesta:

Nessun body richiesto. L'attività è individuata nel path.

Esempio di richiesta: POST /attivita/1

#### Modello di risposta (200 OK):

```
{
  "idAttivita"      : int : 1
  "titolo"          : str : "Pulizia del parco"
  "luogo"           : str : "Parco della Musica, Cagliari"
  "data"            : str : "2026-05-15"
  "oraInizio"       : str : "09:00"
  "oraFine"         : str : "12:00"
  "categoria"       : str : "ambiente"
  "idOrganizzatore" : int : 7
}
```

#### Logica di controllo e vincoli aggiuntivi:

- Validazione orari: oraFine deve essere successiva a oraInizio; il server rifiuta la richiesta in caso contrario.
- Validazione data: la data dell'attività non può essere nel passato rispetto alla data corrente.
- Validazione categoria: il valore deve essere uno tra quelli consentiti; in caso contrario viene restituito 422.

### Domanda 2 - Validazione e gestione errori (5 punti)

#### Descrizione codici di errore:

404 Not Found: la risorsa richiesta non esiste nel sistema. Il server restituisce questo codice quando il client tenta di accedere a un'entità identificata da un ID che non è presente nel database (ad es. un'attività inesistente nella API 3).

422 Unprocessable Entity: la richiesta è sintatticamente corretta ma contiene dati che violano i vincoli logici o di validazione dell'applicazione (ad es. oraFine precedente a oraInizio, categoria non consentita nella API 1).

500 Internal Server Error: errore generico e non previsto lato server, come un malfunzionamento del database o un'eccezione non gestita nel codice. Non è causato da dati errati del client.

#### Esempio di errore 404:

Tentativo di accedere al dettaglio di un'attività inesistente:

GET /attivita/9999

HTTP/1.1 404 Not Found

```
{
  "error": "Activity not found",
  "message": "L'attività con ID 9999 non esiste nel sistema."
}
```

```
}
```

Spiegazione: il client tenta di iscriversi a un'attività che non è presente nel database.

#### **Esempio di errore 422:**

Creazione di un'attività con orario non valido:

POST /attivit 

```
{  
  "titolo": "Pulizia spiaggia",  
  "descrizione": "Raccolta rifiuti",  
  "luogo": "Poetto",  
  "data": "2026-05-15",  
  "oraInizio": "14:00",  
  "oraFine": "09:00",  
  "categoria": "ambiente",  
}
```

HTTP/1.1 422 Unprocessable Entity

```
{  
  "error": "Invalid time range",  
  "message": "L'orario di fine deve essere successivo all'orario di inizio."  
}
```

Spiegazione: oraFine (09:00)   precedente a oraInizio (14:00), violando il vincolo logico specificato.

#### **Domanda 3 - Estensione (3 punti)**

##### **Traccia di risposta:**

I meccanismi di autenticazione sono gi  implementati, per questo motivo sar  sufficiente semplicemente aggiungere nella API di inserimento di una nuova attivit  il controllo di autorizzazione. Non   necessario aggiungere il controllo nelle API di consultazione della lista o del dettaglio delle attivit .